

Learning to Add, Multiply, and Execute Algorithmic Instructions Exactly with Neural Networks

Artur Back de Luca* George Giapitzakis*
 Kimon Fountoulakis

University of Waterloo

{[abackdel](#), [ggiapitz](#), [kimon.fountoulakis](#)}@uwaterloo.ca

Abstract

Neural networks are known for their ability to approximate smooth functions, yet they fail to generalize perfectly to unseen inputs when trained on discrete operations. Such operations lie at the heart of algorithmic tasks such as arithmetic, which is often used as a test bed for algorithmic execution in neural networks. In this work, we ask: can neural networks learn to execute binary-encoded algorithmic instructions exactly? We use the Neural Tangent Kernel (NTK) framework to study the training dynamics of two-layer fully connected networks in the infinite-width limit and show how a sufficiently large ensemble of such models can be trained to execute exactly, with high probability, four fundamental tasks: binary permutations, binary addition, binary multiplication, and Subtract and Branch if Negative (SBN) instructions. Since SBN is Turing-complete, our framework extends to computable functions. We show how this can be efficiently achieved using only logarithmically many training data. Our approach relies on two techniques: structuring the training data to isolate bit-level rules, and controlling correlations in the NTK regime to align model predictions with the target algorithmic executions.

1 Introduction

There has been growing interest in the computational capabilities and efficiency of neural networks both from a theoretical and empirical perspective [Giannou et al. \[2023\]](#), [Kaiser and Sutskever \[2016\]](#), [McLeish et al. \[2024\]](#), [Pérez et al. \[2021\]](#), [Saxton et al. \[2019\]](#), [Siegelmann and Sontag \[1995\]](#). Most works have either demonstrated the expressive power of different architectures through simulation results or learnability from a probably approximately correct (PAC) viewpoint. While important, the mere existence of parameter configurations that realize a specific computation or a generalization bound does not offer insight into the ability to learn to execute an algorithm through gradient-based training. In fact, simulating algorithmic instructions with neural networks often involves approximating discontinuous functions that gradient descent is difficult to converge to with standard training datasets [\[Back De Luca and Fountoulakis, 2024\]](#).

By modeling training with the Neural Tangent Kernel (NTK), we prove that two-layer fully connected networks in the infinite-width limit can learn to iteratively execute *binary permutations*, *binary addition*, *binary multiplication*, and *SBN instructions* with a logarithmic number of examples, or equivalently, a number of examples polynomial in the input bit size. Rather than training on traditional input-output pairs, we exploit the locality of these algorithms by casting each step as a set of templates. We show that training with these local templates is sufficient for full algorithmic execution when composed across iterations of a loop. To our knowledge, this is the first NTK-based proof of exact learnability for these tasks. A high-level overview of our approach is shown in [Figure 1](#).

*Equal contribution.

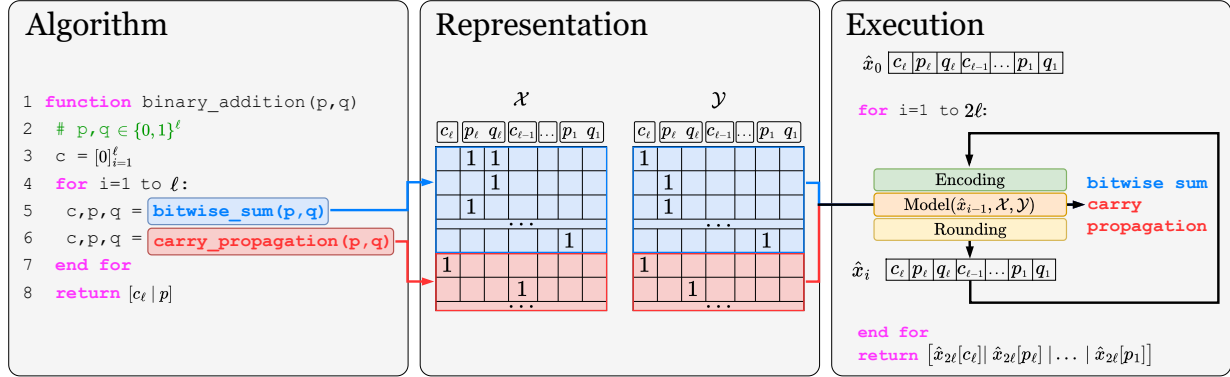


Figure 1: Simplified illustration of the framework used in our analysis. The left panel shows an example algorithm (binary addition) where each function, highlighted in blue and red, is translated into binary training instructions shown in the central panel with matching colors. Each instruction specifies a condition over part of the current algorithm state and maps it to a corresponding output. Instructions are grouped into blocks, indicated by boxed column labels in $\mathcal{X}$ and $\mathcal{Y}$, each representing a subset of the input state. For binary addition, some blocks represent segments of the summands, while others reflect the carry state. For the applications discussed in Section 6, this block structure allows the number of instructions in $\mathcal{X}$ and $\mathcal{Y}$ to scale linearly with bit length ℓ . The right-most panel shows how instructions are used within an iterative framework to update the state vector $\hat{x}_i$, which serves as input to the neural network at the i -th step. The state is first encoded, as described in Section 5, before being passed to the model. In the NTK regime, we show that the network performs template matching against training samples to execute the appropriate instructions. As $\hat{x}_i$ evolves, it activates new templates, progressing through the algorithm. Predictions are rounded at each step to mitigate noise, and repeating this process reproduces the algorithm’s full execution.

Contributions. Our approach is built on two key innovations that overcome the interference and ambiguity in training data that can usually create problems for neural learning for discrete tasks:

1. **Algorithmic template representation:** We demonstrate how to design training data that represent local computations (i.e., operating only on a subset of bits) that can be composed to execute complete algorithms. Each algorithmic instruction is represented by “templates” and entire algorithms (e.g., binary permutations, binary addition, binary multiplication, and SBN instruction execution) can be executed by iteratively matching these templates. The total number of templates is logarithmic in the number of all possible inputs.
2. **Provable exact learnability:** We prove that, by training on an orthonormalized version of our templates, we can control unwanted correlations in the NTK regime and show that an ensemble of two-layer fully connected networks in the infinite-width limit can learn to execute algorithmic instructions exactly with high probability.

2 Literature review

The NTK framework [Jacot et al., 2018] provides insight into continuous-time gradient descent (gradient flow) in fully connected feed-forward neural networks. It has been extended to discrete gradient descent [Lee et al., 2019] and generalized to other architectures like recurrent neural networks (RNNs) and Transformers Yang [2020], Yang and Littwin [2021]. Although NTK theory has been widely developed, few works offer

task-specific guarantees. Notable examples include Boix-Adserà et al. [2024], which proves that Transformers can generalize to unseen symbols for a class of pattern-matching language tasks. Our work addresses a different setting: we show that shallow feed-forward neural networks in the infinite-width limit trained by gradient descent can learn to exactly execute algorithmic instructions using a logarithmic number of examples.

Complementing NTK-based results, other studies have demonstrated the expressive power of neural architectures by simulating algorithmic tasks. Following this approach, Siegelmann and Sontag [1995] proves that RNNs are Turing-complete, and Hertrich and Skutella [2023] demonstrates that RNNs can solve the shortest path problem and approximate solutions to the knapsack problem. Similarly, simulation results on Transformers also establish Turing completeness [Pérez et al., 2021] and present constructive solutions that generalize across input lengths for arithmetic tasks Cho et al. [2025], linear algebra [Giannou et al., 2023, Yang et al., 2024], graph-related problems [Back De Luca and Fountoulakis, 2024], and parallel computation [Back de Luca et al., 2025]. From a learnability perspective, Wei et al. [2022a] provides statistical guarantees for learning Turing-computable functions, while Malach [2023] shows that predictors trained auto-regressively can approximate any such functions. However, these approaches either depend on hand-crafted parameter configurations without training or build upon the PAC framework, and therefore do not address the core question of exact learnability explored in this work.

Learning to execute algorithms has been the focus of numerous empirical studies. These works explore architectural modifications and prompting techniques, particularly aimed at improving generalization in arithmetic tasks Deng et al. [2024, 2025], Jelassi et al. [2023], McLeish et al. [2024], Mistry et al. [2022], Neelakantan et al. [2016], Nogueira et al. [2021], Power et al. [2022], Reed and de Freitas [2016], Saxton et al. [2019], Wang et al. [2017], Wei et al. [2022b], Zaremba and Sutskever [2014]. In this context, Kaiser and Sutskever [2016] introduced the Neural GPU, which learns binary addition and multiplication and generalizes to sequences longer than those seen in training. Similarly, Trask et al. [2018] proposed the Neural Arithmetic Logic Unit (NALU), which incorporates arithmetic operations into network modules to improve generalization. This approach was further refined in Madsen and Johansen [2020] with the introduction of the Neural Addition Unit (NAU) and Neural Multiplication Unit (NMU), offering better stability and convergence. More recent work aims to enhance the length generalization capabilities of Transformers through improved Positional Encodings Jelassi et al. [2023], Ruoss et al. [2023], Shen et al. [2023], Zhou et al. [2024], the use of scratchpads Nye et al. [2022], and prompting techniques for large language models (LLMs), such as Chain-of-Thought (CoT) programming Deng et al. [2024, 2025]. These empirical efforts provide practical methods to improve both in-distribution and out-of-distribution performance of neural networks on arithmetic tasks. However, they lack formal guarantees regarding the conditions under which generalization occurs. Our theoretical analysis offers precise sufficient criteria under which a neural network in the infinite-width limit can provably learn to execute algorithmic instructions.

3 Notation and preliminaries

Throughout the text, we use boldface to denote vectors. The symbols $\mathbf{1}$ and $\mathbf{0}$ denote the vector of all ones and zeros of appropriate length, respectively. We use the notation $[n]$ to refer to the set $\{1, 2, \dots, n\}$. For a vector $\mathbf{x} \in \mathbb{R}^n$ we denote by $\|\mathbf{x}\| := \sqrt{\sum_{i=1}^n x_i^2}$ the Euclidean norm of $\mathbf{x}$. We denote the $n \times n$ identity matrix by I_n .

Model and NTK Results. We provide an overview of the theory used to derive our results. We refer the reader to Golikov et al. [2022] for a comprehensive treatment of the NTK theory. We work with two-layer, ReLU-activated fully connected feed-forward neural networks with no bias. Concretely, the architecture is defined as the function $F : \mathbb{R}^{k'} \rightarrow \mathbb{R}^k$ with $F(\mathbf{x}) = W^2 \text{ReLU}(W^1 \mathbf{x})$ where $W^1 \in \mathbb{R}^{n_h \times k'}$, $W^2 \in \mathbb{R}^{n_k \times k}$, and $n_h \in \mathbb{N}$ is the hidden dimension. The weights are initialized according to the NTK parametrization as $W_{ij}^1 = \frac{\sigma_{\omega}}{\sqrt{k'}} \omega_{ij}^1$ and $W_{ij}^2 = \frac{\sigma_{\omega}}{\sqrt{n_h}} \omega_{ij}^2$ where ω_{ij}^1 and ω_{ij}^2 are trainable parameters initialized i.i.d. from a standard Gaussian distribution. When $n_h \rightarrow \infty$, the empirical NTK kernel given by $\nabla_{\{W^1, W^2\}} F(\mathbf{x})^\top \nabla_{\{W^1, W^2\}} F(\mathbf{x}')$

converges to the deterministic limit:

$$\Theta(\mathbf{x}, \mathbf{x}') = \left(\frac{\mathbf{x}^\top \mathbf{x}'}{2\pi k'} (\pi - \theta) + \frac{\|\mathbf{x}\| \cdot \|\mathbf{x}'\|}{2\pi k'} ((\pi - \theta) \cos \theta + \sin \theta) \right) I_k \in \mathbb{R}^{k \times k} \quad (1)$$

and the NNGP kernel is given by

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = \left(\frac{\|\mathbf{x}\| \cdot \|\mathbf{x}'\|}{2\pi k'} ((\pi - \theta) \cos \theta + \sin \theta) \right) I_k \in \mathbb{R}^{k \times k}, \quad (2)$$

where $\theta = \arccos(\mathbf{x}^\top \mathbf{x}' / \|\mathbf{x}\| \cdot \|\mathbf{x}'\|)$. For a set of vectors $\mathcal{X}$, we will use the notation $\Theta(\mathcal{X}, \cdot)$, $\Theta(\cdot, \mathcal{X})$ and $\Theta(\mathcal{X}, \mathcal{X})$ to refer to the limit NTK calculated when the the set $\{F(\mathbf{x}) : \mathbf{x} \in \mathcal{X}\}$ is vectorized (the outputs are stacked vertically), and similarly for the NNGP kernel. Our learnability results rely on the following theorem by Lee et al. [2019], adapted to our architecture:

Theorem 3.1 (Theorem 2.2 from Lee et al. 2019). *Let $\mathcal{X}$ and $\mathcal{Y}$ be the training dataset (training inputs and ground truth labels, respectively). Assume that $\Theta := \Theta(\mathcal{X}, \mathcal{X})$ is positive definite. Suppose the network is trained with gradient descent (with small-enough step-size) or gradient flow to minimize the empirical MSE loss.¹ Then, for every $\hat{\mathbf{x}} \in \mathbb{R}^{k'}$ with $\|\hat{\mathbf{x}}\| \leq 1$, as $n_h \rightarrow \infty$, the output at training time t , $F_t(\hat{\mathbf{x}})$, converges in distribution to a Gaussian with mean and variance given by*

$$\mu(\hat{\mathbf{x}}) = \Theta(\hat{\mathbf{x}}, \mathcal{X}) \Theta^{-1} \mathcal{Y} \quad (3)$$

$$\Sigma(\hat{\mathbf{x}}) = \mathcal{K}(\hat{\mathbf{x}}, \hat{\mathbf{x}}) + \Theta(\hat{\mathbf{x}}, \mathcal{X}) \Theta^{-1} \mathcal{K}(\mathcal{X}, \mathcal{X}) \Theta^{-1} \Theta(\mathcal{X}, \hat{\mathbf{x}}) - (\Theta(\hat{\mathbf{x}}, \mathcal{X}) \Theta^{-1} \mathcal{K}(\mathcal{X}, \hat{\mathbf{x}}) + h.c.) \quad (4)$$

where $\mathcal{Y}$ in Equation (3) denotes the vectorization of all vectors $\mathbf{y} \in \mathcal{Y}$, and “h.c.” is an abbreviation for the Hermitian conjugate.

4 Algorithmic execution via template matching

In this section, we introduce a template matching principle that offers a high-level intuition for representing and executing algorithmic instructions in the NTK regime. This principle transforms binary inputs into binary outputs by comparing input configurations against a set of predefined templates. Matched templates are then used to compose the corresponding output. We use this principle of templates and template matching functions to encode and execute algorithms. While this approach may seem unrelated to neural networks, we show in the following sections that, by carefully structuring training and test inputs, neural networks can emulate this template matching mechanism. In doing so, they can learn algorithms on binary data through the lens of the Neural Tangent Kernel. More concretely, we show that this principle, as described here, allows us to learn and express arithmetic operations such as addition and multiplication. Furthermore, this approach can be generalized to more general computations, as discussed in Section 5.

Input representation: Inputs are vectors of binary variables of length k , i.e., $\hat{\mathbf{x}} \in \{0, 1\}^k$. These variables are grouped into disjoint blocks, partitioning $\hat{\mathbf{x}}$ into b blocks. Each block, denoted by B_i for $i \in [b]$, has a length $s_i \in \mathbb{N}$. While block sizes may vary, each s_i is assumed to be $\mathcal{O}(1)$. Let $\hat{\mathbf{x}}[B_i] \in \{0, 1\}^{s_i}$ denote the subvector for block B_i , i.e., the bits of $\hat{\mathbf{x}}$ that belong to that block.

Templates and functions: The transformation of the input vector depends on the configurations present within each block. Each block B_i is associated with a finite set of templates $\mathcal{T}_i \subseteq \{0, 1\}^{s_i} \times \{0, 1\}^k$, where each template $(\mathbf{x}, \mathbf{y})$ maps a block configuration $\mathbf{x}$ to a complete output vector $\mathbf{y}$. The mapping is functional: for all $(\mathbf{x}, \mathbf{y}), (\mathbf{x}', \mathbf{y}') \in \mathcal{T}_i$, if $\mathbf{x} = \mathbf{x}'$, then $\mathbf{y} = \mathbf{y}'$. In other words, no block configuration maps to more than one output. Consequently, the cardinality of $\mathcal{T}_i$ is at most 2^{s_i} .

Using each $\mathcal{T}_i$, we define a block-specific pattern matching function $f_i : \{0, 1\}^{s_i} \rightarrow \{0, 1\}^k$ and an aggregation function $f : \{0, 1\}^k \rightarrow \{0, 1\}^k$ by:

¹The empirical MSE loss is defined as $\mathcal{L}(\mathcal{D}) = (2|\mathcal{D}|)^{-1} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}} \|f_t(\mathbf{x}) - \mathbf{y}\|^2$.

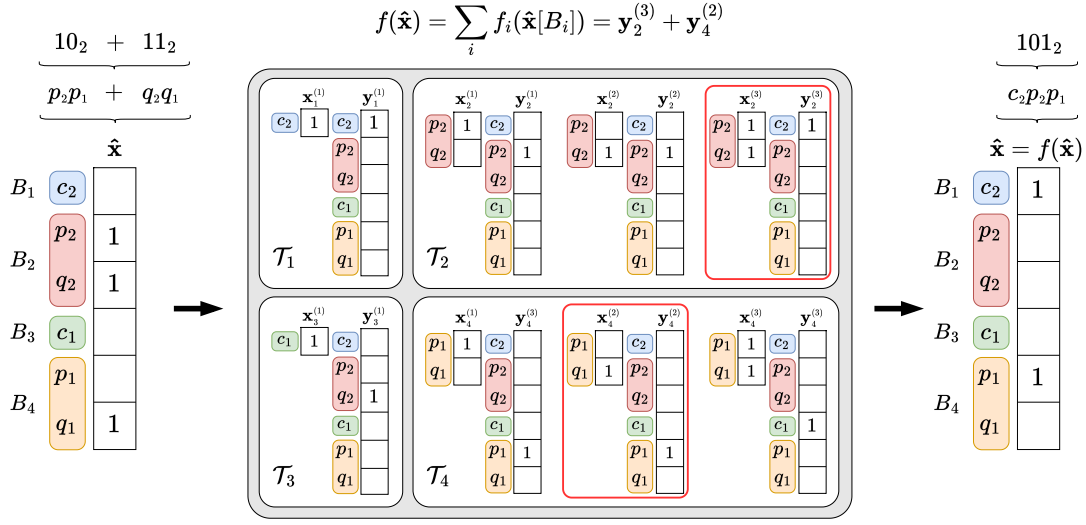


Figure 2: Illustration of the addition algorithm based on the template matching approach from Section 4. Two $\ell = 2$ bit numbers, $p = 2$ (or 10_2) and $q = 3$ (or 11_2), are added by organizing their bits and carries into blocks B_i . Blocks B_2 and B_4 represent the bits of p and q , while B_1 and B_3 handle the carries. The input $\hat{\mathbf{x}}$ is processed via template matching f , using templates $\mathcal{T}_i$, producing outputs $y_i^{(k)}$ used to compose the output. Although the method is iterative, this example completes in one step. The final result 5 (or 101_2) is stored at the most-significant carry bit and the bits of p in $\hat{\mathbf{x}}$.

$$f_i(\mathbf{x}') := \begin{cases} \mathbf{y} & \text{if } (\mathbf{x}', \mathbf{y}) \in \mathcal{T}_i \\ \mathbf{0} & \text{otherwise,} \end{cases} \quad (5)$$

$$f(\hat{\mathbf{x}}) := \bigvee_{i=1}^b f_i(\hat{\mathbf{x}}[B_i]) \quad (6)$$

where $\mathbf{0}$ denotes the all-zero vector in $\{0, 1\}^k$, and the bitwise disjunction (logical OR) is applied elementwise across the output vectors $f_i(\hat{\mathbf{x}}[B_i])$ inside Equation (6). Each of these template matching functions is applied independently and simultaneously to the corresponding block.

In this framework, the block-specific templates $\mathcal{T}_i$ determine the local behavior of the algorithm, specifying how each block contributes to the global state vector $\hat{\mathbf{x}}$. The global update function f , formed by aggregating the output of all f_i , enforces this behavior across all blocks simultaneously. By applying f iteratively, we propagate these local rules over time, effectively executing the algorithm.

4.1 Algorithmic example: computing binary addition

We now demonstrate how to apply the template matching principle to simulate binary addition. Throughout this example – and the more formal algorithm descriptions provided in Appendix B – we often assign descriptive variable names to improve clarity. These identifiers serve only as labels and do not affect computation. For this example, we denote the two summands by p and q , each consisting of $\ell = 2$ bits. Consequently, their sum requires at most $\ell + 1 = 3$ bits to be represented.

The addition algorithm emulates a ripple-carry adder built from half-adders [Harris and Harris, 2012], performing bitwise addition, while propagating carries to higher-order bits. The process alternates between ℓ summation and ℓ carry-propagation steps, reaching a steady state after at most 2ℓ iterations. However, as demonstrated in Appendix B.2, it is also possible to introduce a flag to indicate termination.

To simulate this behavior, the input structure is organized into blocks corresponding to the bits of p and

q , along with their associated carry bits. In this example, we use four blocks: two for the individual bits p_i and q_i , and two others for the carries, denoted c_i , for $i = 1, 2$. In our representation, the final output comprises the most significant carry bit, followed by the bits of p as stored in $\hat{x}$. As shown in Figure 2, the result is formed by concatenating c_2 , p_2 , and p_1 , for a total of $\ell + 1 = 3$ output bits. To capture the required operations in each block, we define a set of templates, as illustrated in Figure 2. The even-numbered templates implement bitwise summation, while the odd-numbered templates handle carry propagation to the next bit. These templates are designed to ensure that the operations proceed without interfering with one another.

5 Exact learnability Part I: NTK predictor behavior

In this section, we analyze the exact learnability of algorithmic executions in neural networks by studying the NTK predictor, defined as the mean of the limiting distribution for a two-layer network. We show that it preserves sign-based information about the ground truth and can learn to execute algorithms framed as template matching, following the framework in Section 4. The training dataset is built from templates and, in the applications presented, its size scales with the number of bits, hence logarithmic in the number of possible binary inputs.

5.1 Input specification

We begin our analysis by specifying the structure of the training and testing inputs. Following the framework of Section 4, we construct the training inputs as block-partitioned vectors, where each block corresponds to an input template set. Each training input is non-zero only within the block determined by its associated input template in $\mathcal{T}_i$. In contrast, test inputs may contain multiple non-zero entries, each corresponding to a configuration appearing in the dataset. A visualization of the input configuration is provided in Figure 3.

Training dataset We define the training set for any algorithmic task described as in Section 4. Let each of the b block configurations $\mathcal{T}_i$ be a set of $t_i := |\mathcal{T}_i|$ template-label tuples, i.e., each

$$\mathcal{T}_i = \{(\mathbf{x}^{(i,1)}, \mathbf{y}^{(i,1)}), \dots, (\mathbf{x}^{(i,t_i)}, \mathbf{y}^{(i,t_i)})\} \subseteq \{0, 1\}^{s_i} \times \{0, 1\}^k.$$

The input dimension to the neural network is $k' = \sum_{i=1}^b t_i$.² We view $\mathbb{R}^{k'}$ as the direct sum $\mathbb{R}^{t_1} \oplus \dots \oplus \mathbb{R}^{t_b}$ and for each subspace $\mathbb{R}^{t_i}$, we choose an orthonormal basis $\{\mathbf{u}_{i1}, \dots, \mathbf{u}_{it_i}\}$. For each $i = 1, \dots, b$, we encode the j -th template of $\mathcal{T}_i$, $(\mathbf{x}^{(i,j)}, \mathbf{y}^{(i,j)})$, as the block-partitioned vector

$$\mathbf{q}_{ij} = (\mathbf{0}_{t_1}, \dots, \mathbf{0}_{t_{i-1}}, \mathbf{u}_{ij}, \dots, \mathbf{0}_{t_b})^\top \in \mathbb{R}^{k'}$$

with the i -th block being equal to $\mathbf{u}_{ij}$. The sets of training inputs and corresponding ground-truth labels are given by

$$\mathcal{X} = \{\mathbf{q}_{ij} : i \in [b], j \in [t_i]\} \subseteq \mathbb{R}^{k'} \quad \text{and} \quad \mathcal{Y} = \{\mathbf{y}^{(i,j)} : i \in [b], j \in [t_i]\} \subseteq \mathbb{R}^k,$$

respectively. This encoding yields an orthonormal, block-partitioned set of training inputs, with no cross-block interference. In total, the dataset has size $k' \in \mathcal{O}(b)$, and b depends on the number of bits used for number representation in each application. For example, to add two 10-bit numbers, 40 training examples are required.

Test inputs Based on the algorithmic execution framework of Section 4 every test input $\hat{x}$ is expressed as $\hat{x} = \frac{1}{\sqrt{n_{\hat{x}}}}(\hat{x}_1, \dots, \hat{x}_b)^\top$ with each $\hat{x}_i$ being either $\mathbf{0}_{t_i}$ or matching one of the training samples of $\mathcal{X}$ in its i -th block, i.e. $\hat{x}_i$ is equal to the i -th block of $\mathbf{q}_{ij}$ for some $j \in [t_i]$. In that case, we say that $\hat{x}$ matches $\mathbf{q}_{ij}$. We denote by $n_{\hat{x}}$ the number of blocks of $\hat{x}$ that match an entry of the training set. Since each t_i is $\mathcal{O}(1)$,

²In applications such as binary multiplication and SBN, the dimension k' is extended with auxiliary unitary blocks, each containing a template. These extensions, detailed in Appendix C, ensure Assumption 5.1 is satisfied without affecting the algorithm, as they are never matched during execution.

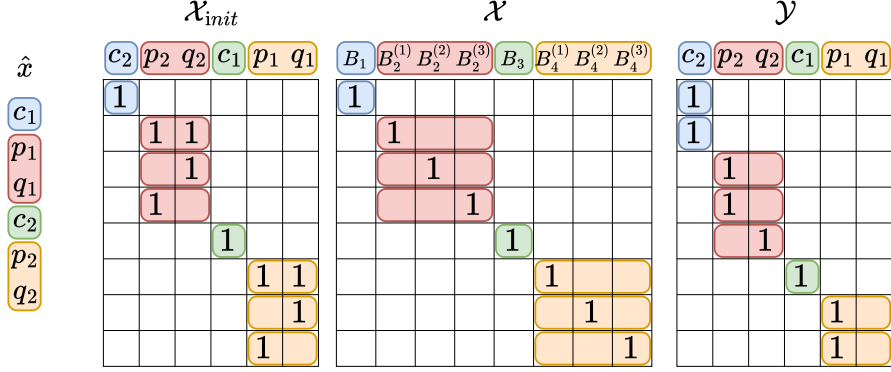


Figure 3: Visualization of the input specification of [Section 5.1](#) for binary summation of two $\ell = 2$ bit numbers. On the left, we illustrate the block structure of a pre-encoded test sample. Each block should either be zero or match the corresponding block of an element (row) of $\mathcal{X}_{\text{init}}$. On the right, we showcase the encoding procedure that creates the training dataset. The initial examples (described in [Section 4](#)) forming the rows of $\mathcal{X}_{\text{init}}$ are augmented and orthogonalized. Notice that the colored parts of each row of $\mathcal{X}_{\text{init}}$ along with the corresponding row of $\mathcal{Y}$ match the $\mathcal{T}_i$'s of [Figure 2](#). Also note that the orthogonalization presented here is only one of the many possible ones. Finally, each row of $\mathcal{Y}$ depicts the corresponding ground-truth output for each training sample.

the total number of test inputs is $\mathcal{O}(2^b)$, which is exponentially larger than the training dataset size. For instance, the total number of test inputs for the addition of two 10-bit numbers is 4^{10} , which is exponentially larger than the training dataset size.

5.2 NTK predictor behavior theorem

To derive this result, we introduce and discuss one additional assumption. The specific structure of our orthogonal training set and test inputs simplifies the mean of the limiting distribution, $\mu(\hat{\mathbf{x}}) = \Theta(\hat{\mathbf{x}}, \mathcal{X})\Theta(\mathcal{X}, \mathcal{X})^{-1}\mathcal{Y}$. This predictor can be expressed as a simple weighted sum of the ground-truth training labels $\mathcal{Y}$. This sum is governed by two distinct weights: a “signal” weight, $w^1 \equiv w^1(\hat{\mathbf{x}})$, applied to training labels whose corresponding inputs match a block in the test input $\hat{\mathbf{x}}$, and an “interference” weight, $w^0 \equiv w^0(\hat{\mathbf{x}})$, applied to all labels from unmatched training inputs. For any given output bit, an “unwanted correlation” occurs when an unmatched training sample (which receives the w^0 weight) also has that bit set, thus contributing interference against the correct signal. Informally, we can interpret the ratio $-w^1/w^0$ as a decision margin. Our assumption, stated below, simply requires that the total number of these unwanted correlations is less than this margin, ensuring the signal’s contribution outweighs the total interference. We now formally state the assumption that guarantees learnability:

Assumption 5.1. For each test input $\hat{\mathbf{x}}$ and for each position $i \in [k']$ such that the ground-truth output $f(\hat{\mathbf{x}})^3$ has the i -th bit set, the number of training examples that do not match $\hat{\mathbf{x}}$ and have the i -th bit set (which we call *unwanted correlations*) is less than the ratio $-w^1(\hat{\mathbf{x}})/w^0(\hat{\mathbf{x}})$.⁴

While, at first, [Assumption 5.1](#) may seem restrictive, we remark that for many algorithms, including those examined in this work, the number of conflicts is low enough to guarantee that it is satisfied. In particular,

³There is a slight abuse of notation here when using $f(\hat{\mathbf{x}})$ since f does not operate on encoded inputs. Depending on the context, we may use $\hat{\mathbf{x}}$ to denote both the pre-encoded and encoded test inputs.

⁴ $w^0(\hat{\mathbf{x}})$ is always non-positive and so the ratio is non-negative.

since the SBN instruction set is Turing-complete, it can, in principle, encode any algorithm. Our framework leverages this property by simulating SBN instructions, allowing any algorithm to be represented as input to the system. This does not mean that we directly learn arbitrary Turing-computable functions, but it ensures that such functions can be expressed within our formulation without violating [Assumption 5.1](#). The behavior of the NTK predictor is given in the following theorem:

Theorem 5.1 (NTK predictor behavior). *Consider an algorithmic problem cast as template-matching and encoded in a training set $(\mathcal{X}, \mathcal{Y}) \subseteq \mathbb{R}^{k'} \times \mathbb{R}^k$ as described in [Section 5.1](#). Then, under [Assumption 5.1](#), the mean of the limiting NTK distribution $\mu(\hat{\mathbf{x}}) = \Theta(\hat{\mathbf{x}}, \mathcal{X})\Theta(\mathcal{X}, \mathcal{X})^{-1}\mathcal{Y}$ for any test input $\hat{\mathbf{x}} \in \mathbb{R}^{k'}$ contains sign-based information about the ground-truth output, namely for each coordinate of the output $i = 1, \dots, k$, $\mu(\hat{\mathbf{x}})_i \leq 0$ if the ground-truth bit at position i , $f(\hat{\mathbf{x}})_i$, is set, and $\mu(\hat{\mathbf{x}})_i > 0$ if the ground-truth bit at position i , $f(\hat{\mathbf{x}})_i$, is not set.*

Proof outline. We aim to express each $\mu(\hat{\mathbf{x}})_i$ as a weighted bit-sum with weights w^0 and w^1 , and then rely on [Assumption 5.1](#) to guarantee that the signs are preserved. The orthogonality of the training dataset forces the train NTK to align with the local computation structure of the template matching framework. In particular, both the train and test NTK kernels assume a scaled identity that keeps different blocks from interfering with one another. Furthermore, since any test input activates at most one vector per block, the test diagonal elements of the test NTK kernel (measuring the similarity of the test input to each element of the training set) take only two possible values: one indicating “this block is active” and one indicating “this block is empty”. Concretely, $\Theta^{-1} := \Theta(\mathcal{X}, \mathcal{X})^{-1}$ takes the form $\tilde{\Theta}^{-1} \otimes I_k$ where $\tilde{\Theta}^{-1} \in \mathbb{R}^{k' \times k'}$ is a scaled identity plus a rank-1 perturbation (noise), and $\Theta(\hat{\mathbf{x}}, \mathcal{X})$ takes the form $\mathbf{f}^\top \otimes I_k$ where $\mathbf{f} \in \mathbb{R}^{k'}$ takes only two values, f^1 and f^0 denoting match/no-match. By arranging the elements of $\mathcal{Y}$ as columns in a matrix Y and using vectorization, we can rewrite the model as

$$\mu(\hat{\mathbf{x}}) = \Theta(\hat{\mathbf{x}}, \mathcal{X})\Theta^{-1}\mathcal{Y} = ((\mathbf{f}\tilde{\Theta}^{-1}) \otimes I_k) \text{vec}(Y) = Y\tilde{\Theta}^{-1}\mathbf{f}^\top.$$

We first show that $\tilde{\Theta}^{-1}\mathbf{f}^\top \in \mathbb{R}^{k'}$ takes only two values $w^0 \leq 0$ and $w^1 > 0$ depending on whether the corresponding entry in $\mathbf{f}$ is equal to f^0 or f^1 . To conclude, we write

$$\mu(\hat{\mathbf{x}})_i = \sum_{(j,l) \in \mathcal{I}_+(\hat{\mathbf{x}})} f(\mathbf{q}_{jl})_i w^1 + \sum_{(j,l) \in \mathcal{I}_-(\hat{\mathbf{x}})} f(\mathbf{q}_{jl})_i w^0, \quad (7)$$

where

$$\mathcal{I}_+(\hat{\mathbf{x}}) = \{(j, l) : j \in [m], l \in [s_j] \text{ and } \hat{\mathbf{x}} \text{ matches } \mathbf{q}_{jl}\},$$

and

$$\mathcal{I}_-(\hat{\mathbf{x}}) = \{(j, l) : j \in [m], l \in [s_j] \text{ and } \hat{\mathbf{x}} \text{ doesn't match } \mathbf{q}_{jl}\}.$$

The sets $\mathcal{I}_+(\hat{\mathbf{x}})$ and $\mathcal{I}_-(\hat{\mathbf{x}})$ partition the training dataset into two disjoint sets: the indices of the training dataset that match $\hat{\mathbf{x}}$ and the ones that do not. When $f(\hat{\mathbf{x}})_i = 0$, [Equation \(6\)](#) yields a vanishing first summation and therefore $\mu(\hat{\mathbf{x}})_i \leq 0$. On the other hand, when $f(\hat{\mathbf{x}})_i = 1$, due to the fact that $\hat{\mathbf{x}}$ matches exactly one of the training samples from the block containing the i -th bit, [Equation \(7\)](#) reduces to $\mu(\hat{\mathbf{x}})_i = w^1 + |\mathcal{I}_-^1(\hat{\mathbf{x}})| \cdot w^0$, where $\mathcal{I}_-^1(\hat{\mathbf{x}}) = \{(j, l) \in \mathcal{I}_-(\hat{\mathbf{x}}) : f(\mathbf{q}_{jl})_i = 1\}$ denotes the index set of unwanted correlations. Under [Assumption 5.1](#), we have $\mu(\hat{\mathbf{x}})_i > 0$ and so the sign-based ground truth information is preserved, concluding the proof. A pictorial version of the above outline is given in [Figure 4](#).

□

Regarding the algorithms discussed throughout this paper, we have the following remark⁵:

Remark 5.1. *The tasks of binary permutations, binary addition, binary multiplication, and executing SBN instructions all satisfy the assumptions of [Theorem 5.1](#).*

⁵Our results for permutation, addition, and multiplication were numerically verified on various random instances by calculating the corresponding limiting mean. For code and implementation details, refer to the supplementary material.

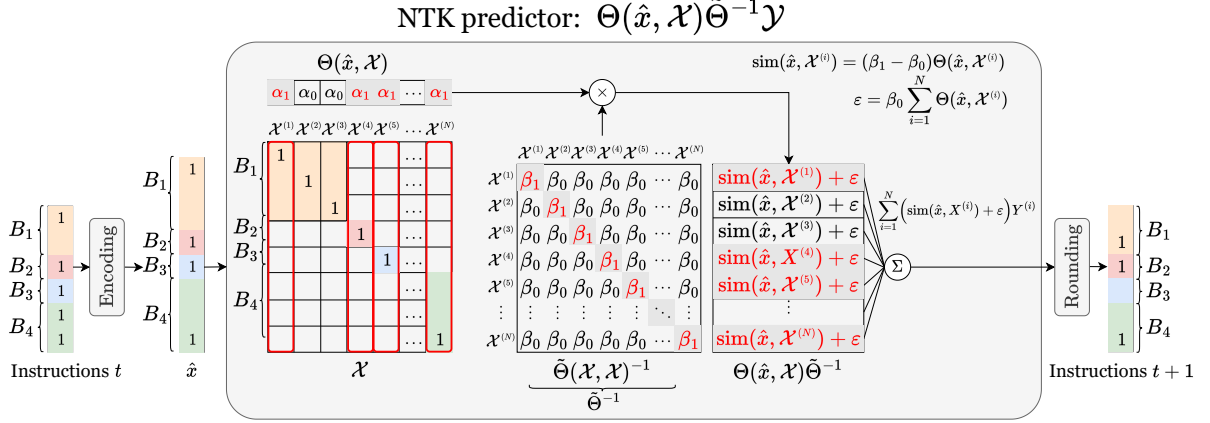


Figure 4: Illustration of the NTK predictor structure: inputs are first encoded (normalization omitted) to compute the test NTK $\Theta(\hat{x}, \mathcal{X})$. Due to the test input structure, this kernel assumes two values based on matches between test and training inputs within blocks. Multiplying by $\tilde{\Theta}^{-1}$ (which assumes the form of scaled identity plus a rank-1 noise perturbation colored black) re-weights these similarities, and the multiplication by $\mathcal{Y}$ gives the final prediction. When the contribution of the unmatched entries is controlled (black similarities), the sign of each coordinate matches the ground-truth output.

The proof strategy for Remark 5.1 involves computing a lower bound for the decision margin $-w^1(\hat{x})/w^0(\hat{x})$ over all test inputs $\hat{x}$ and showing that the number of unwanted correlations falls below this minimum value. For example, in the case of binary addition, this threshold comes out to be equal to 4 while the number of unwanted correlations can be at most 1. In Figure 3, this is captured by the fact that each column of $\mathcal{Y}$ contains at most 2 ones. A complete proof of Theorem 5.1 and Remark 5.1 can be found in Appendix C.

6 Exact learnability Part II: high-probability guarantee

In this section, we continue our proof of exact learning of algorithmic instructions using neural networks. The conclusion of Theorem 5.1 suggests a simple procedure: for each coordinate, if the output of the network is greater than zero, round to 1, otherwise round to 0. To extend this result from the NTK predictor to actual models and ensure high-probability guarantees of exact learning, we can independently train enough models, average their outputs, and round accordingly.⁶ We define *ensemble complexity* as the number of models required to achieve a desired level of post-rounding accuracy. In what follows, we derive a lower bound on the ensemble complexity of learning algorithmic instructions and give its asymptotic order. This completes the proof that neural networks can, with high probability, exactly learn algorithmic instructions.

Given a test input $\hat{x}$ with ground truth $\hat{y} = f(\hat{x})$, let $F^j(\hat{x})$ be the output of the j -th model in an ensemble of N independently trained networks. By Theorem 3.1, each $F^j(\hat{x})$ is drawn i.i.d. from $\mathcal{N}(\mu(\hat{x}), \Sigma(\hat{x}))$, so every coordinate $F^j(\hat{x})_i$ follows $\mathcal{N}(\mu(\hat{x})_i, \sigma^2(\hat{x}))$.⁷ Define the ensemble mean $G(\hat{x}) = \frac{1}{N} \sum_{j=1}^N F^j(\hat{x})$. Because $\mu(\hat{x})_i \leq 0$ when $\hat{y}_i = 0$ and $\mu(\hat{x})_i > 0$ when $\hat{y}_i = 1$, rounding $G(\hat{x})_i$ is correct if $|G(\hat{x})_i - \mu(\hat{x})_i| < |\mu(\hat{x})_i|/2$.

Applying a standard Gaussian concentration bound given in Lemma A.1 and the union bound, we obtain, for any $\delta \in (0, 1)$, perfect post-rounding accuracy with probability $1 - \delta$ whenever the number of averaged

⁶In practice, model-to-model variability is often controlled by training several copies (or by collecting multiple checkpoints along one training run) and then averaging either their predictions Lakshminarayanan et al. [2017] or their weights Izmailov et al. [2018]. Our “ensemble complexity” result gives a clean theoretical analogue of this variance-reduction trick, with concrete high-probability guarantees on post-rounding accuracy.

⁷See Appendix D for the calculation of $\sigma^2(\hat{x})$.

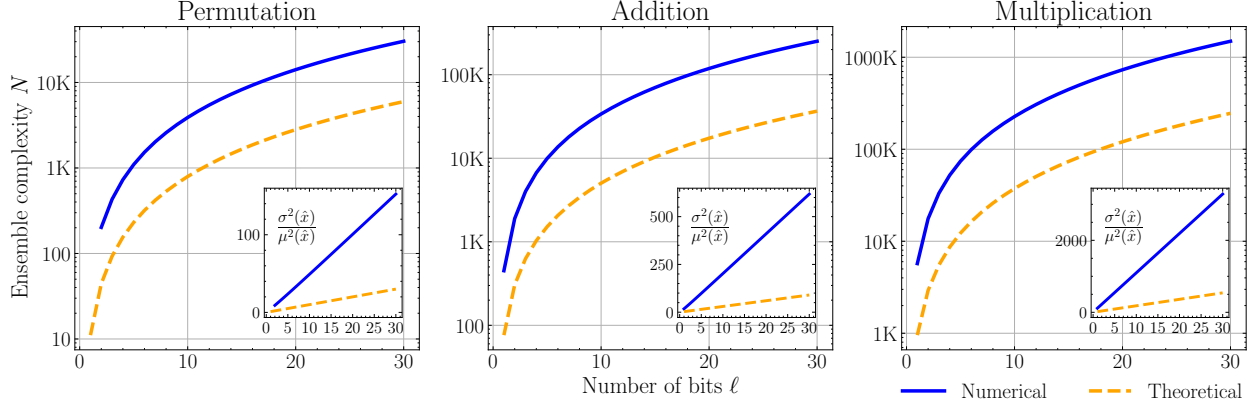


Figure 5: Numerical and theoretical estimates of ensemble complexity N (in log-scale) for permutation, addition, and multiplication tasks as a function of bit length ℓ . Ensemble complexity is computed via a union bound over all possible inputs and algorithmic executions for a given ℓ . Inset blocks illustrate the ratio of variance to mean in Equation (8), estimated using input size k' . This ratio increases linearly with k' up to a constant. The same ratio is used in the theoretical estimate of N , which matches the numerical estimate in growth rate, differing only by a constant factor.

models N satisfies:

$$N \geq 8 \max_{\hat{\mathbf{x}}, i \in [k]} \left\{ \frac{\sigma^2(\hat{\mathbf{x}})}{\mu^2(\hat{\mathbf{x}})_i} \right\} \ln \left(\frac{2k'}{\delta} \right). \quad (8)$$

To derive the order of the bound on the ensemble complexity of Equation (8), we need to analyze the asymptotic orders of $\mu(\hat{\mathbf{x}})$ and $\sigma^2(\hat{\mathbf{x}})$. The result is summarized in the following technical lemma:

Lemma 6.1. *Suppose we train on any of the four tasks (permutations, addition, multiplication, SBN instructions), as described in Section 5. Let $\hat{\mathbf{x}}$ be a test input unseen during training matching $n_{\hat{\mathbf{x}}} > 1$ training entries and let $\mu(\hat{\mathbf{x}})$ and $\sigma^2(\hat{\mathbf{x}})$ be as in Theorem 3.1. Then $\sigma^2(\hat{\mathbf{x}}) \in \mathcal{O}(1/k')$, and, for each $i \in [k]$, depending on the relationship between $n_{\hat{\mathbf{x}}}$ and k' we have:*

$$|\mu(\hat{\mathbf{x}})_i| \in \begin{cases} \Theta\left(\frac{1}{k'}\right) & \text{if } n_{\hat{\mathbf{x}}} \text{ is const.} \\ \Theta\left(\frac{\sqrt{n_{\hat{\mathbf{x}}}}}{k'}\right) & \text{if } n_{\hat{\mathbf{x}}} \text{ is non-const.} \\ \Theta\left(\frac{1}{\sqrt{k'}}\right) & \text{if } n_{\hat{\mathbf{x}}} = ck' \text{ for some } c \in (0, 1] \end{cases} \quad \text{and} \quad |\mu(\hat{\mathbf{x}})_i| \in \begin{cases} \Theta(1) & \text{if } n_{\hat{\mathbf{x}}} \text{ is const.} \\ \Theta\left(\frac{1}{\sqrt{n_{\hat{\mathbf{x}}}}}\right) & \text{if } n_{\hat{\mathbf{x}}} \text{ is non-const.} \\ \Theta\left(\frac{1}{\sqrt{k'}}\right) & \text{if } n_{\hat{\mathbf{x}}} = ck' \text{ for some } c \in (0, 1) \end{cases}$$

when the ground-truth bit at position i is not set ($f(\hat{\mathbf{x}})_i = 0$) or set ($f(\hat{\mathbf{x}})_i = 1$), respectively.

The proof of Lemma 6.1 (including the calculation of $\sigma^2(\hat{\mathbf{x}})$) can be found in Appendix D. In light of these asymptotic results, the uniform bound of Equation (8) behaves like $\mathcal{O}(k' \log k')$. An application of the union bound shows that for an algorithm requiring m iterations, the ensemble complexity bound when accounting for all $\mathcal{O}(2^b)$ possible test inputs (where b is the number of templates as in Section 4) and all m iterations behaves like $\mathcal{O}(k'b + k' \log m)$. In particular, for the tasks considered, we have the following remark:

Remark 6.1. For the tasks of binary permutation ($b = k' = \ell$, $m = 1$), binary addition ($b = 2\ell$, $k' = 4\ell$, $m = 2\ell$), and binary multiplication ($b = 11\ell$, $k' = 21\ell$, $m = 4\ell^2 + 3\ell$) the ensemble complexity scales like $\mathcal{O}(\ell^2)$, where ℓ is the bit length for each application.

Figure 5 plots numerical and theoretical estimates for the ensemble complexity of the permutation, addition, and multiplication tasks, showcasing the conclusion of Remark 6.1. In Appendix E, we verify our theory with an empirical estimate of the ensemble complexity for the permutation task. This is done by training multiple two-layer fully connected feed-forward networks, each with 50,000 hidden units, using full-batch gradient descent to form an ensemble.

7 Limitations and future work

In this work, we have demonstrated that two-layer fully connected feed-forward neural networks in the infinite-width limit can learn to execute algorithmic instructions expressed within our template matching framework (including binary permutations, binary addition, binary multiplication, and execution of SBN instructions) using a training set of logarithmic size in the number of possible binary inputs. This provides an affirmative answer to the question of whether neural networks can learn to execute long sequences of binary-encoded instructions exactly.

Our analysis, however, relies on several simplifying assumptions that bound its generality. The first concerns data orthogonality and explicit instruction access, which guarantee that each local computation step is independently learnable. Future work could investigate whether exact learning remains possible under correlated training examples or when the network must infer primitive instructions from only partial input-output traces. The second limitation arises from the bounded memory setting of our framework. Increasing the available memory changes the input dimensionality of the model and, therefore, requires retraining, so extrapolation to longer inputs does not occur automatically. Within this bounded memory regime, exact algorithmic execution remains achievable using only logarithmically many short training examples, but extending these results to architectures that naturally process variable-length inputs, such as RNNs, Transformers, or GNNs, would be a valuable next step. For example, the use of GNNs on bounded degree graphs may enable controlled forms of length generalization with respect to graph size while preserving the theoretical structure of our exact learning framework.

Acknowledgements

K. Fountoulakis would like to acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC). Cette recherche a été financée par le Conseil de recherches en sciences naturelles et en génie du Canada (CRSNG), [RGPIN-2019-04067, DGEER-2019-00147].

G. Giapitzakis would like to acknowledge the support of the Onassis Foundation - Scholarship ID: F ZU 020-1/2024-2025.

References

- Artur Back De Luca and Kimon Fountoulakis. Simulation of graph algorithms with looped transformers. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 2319–2363. PMLR, 2024. URL <https://dl.acm.org/doi/10.5555/3692070.3692162>.
- Artur Back de Luca, George Giapitzakis, Shenghao Yang, Petar Veličković, and Kimon Fountoulakis. Positional attention: Expressivity and learnability of algorithmic computation, 2025. URL <https://arxiv.org/abs/2410.01686>.

- Enric Boix-Adserà, Omid Saremi, Emmanuel Abbe, Samy Bengio, Etai Littwin, and Joshua M. Susskind. When can transformers reason with abstract symbols? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=STUGfUz8ob>.
- Hanseul Cho, Jaeyoung Cha, Srinadh Bhojanapalli, and Chulhee Yun. Arithmetic transformers can length-generalize in both operand length and count. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=eIgGesYKLG>.
- Yuntian Deng, Kiran Prasad, Roland Fernandez, Paul Smolensky, Vishrav Chaudhary, and Stuart Shieber. Implicit chain of thought reasoning via knowledge distillation, 2024. URL <https://openreview.net/forum?id=9cumTvv1HG>.
- Yuntian Deng, Yejin Choi, and Stuart Shieber. From explicit cot to implicit cot: Learning to internalize cot step by step, 2025. URL <https://openreview.net/forum?id=fRPmc94QeH>.
- Angeliki Giannou, Shashank Rajput, Jy-Yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 11398–11442, 2023. URL <https://dl.acm.org/doi/10.5555/3618408.3618866>.
- William F Gilreath and Phillip A Laplante. *Computer architecture: A minimalist perspective*, volume 730. Springer Science & Business Media, 2003.
- Eugene Golikov, Eduard Pokonechnyy, and Vladimir Korviakov. Neural tangent kernel: A survey, 2022. URL <https://arxiv.org/abs/2208.13614>.
- David Harris and Sarah Harris. *Digital Design and Computer Architecture, Second Edition*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2nd edition, 2012. ISBN 0123944244.
- Christoph Hertrich and Martin Skutella. Provably good solutions to the knapsack problem via neural networks of bounded size. *INFORMS journal on computing*, 35(5):1079–1097, 2023. URL <https://pubsonline.informs.org/doi/10.1287/ijoc.2021.0225>.
- Wolfram Research, Inc. Mathematica, Version 14.1, 2024. URL <https://www.wolfram.com/mathematica>. Champaign, IL.
- Pavel Izmailov, Dmitrii Podoprikin, Timur Garipov, Dmitry P. Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization. In Amir Globerson and Ricardo Silva, editors, *Proceedings of the Thirty-Fourth Conference on Uncertainty in Artificial Intelligence, UAI 2018, Monterey, California, USA, August 6-10, 2018*, pages 876–885. AUAI Press, 2018. URL <http://auai.org/uai2018/proceedings/papers/313.pdf>.
- Arthur Jacot, Franck Gabriel, and Clement Hongler. Neural tangent kernel: Convergence and generalization in neural networks. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/file/5a4be1fa34e62bb8a6ec6b91d2462f5a-Paper.pdf.
- Samy Jelassi, Stéphane d’Ascoli, Carles Domingo-Enrich, Yuhuai Wu, Yuanzhi Li, and François Charton. Length generalization in arithmetic transformers, 2023. URL <https://arxiv.org/abs/2306.15400>.
- Lukasz Kaiser and Ilya Sutskever. Neural gpus learn algorithms. In Yoshua Bengio and Yann LeCun, editors, *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016. URL <http://arxiv.org/abs/1511.08228>.

- Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/9ef2ed4b7fd2c810847ffa5fa85bce38-Paper.pdf.
- Jaehoon Lee, Lechao Xiao, Samuel Schoenholz, Yasaman Bahri, Roman Novak, Jascha Sohl-Dickstein, and Jeffrey Pennington. Wide neural networks of any depth evolve as linear models under gradient descent. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/0d1a9651497a38d8b1c3871c84528bd4-Paper.pdf.
- Andreas Madsen and Alexander Rosenberg Johansen. Neural arithmetic units. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=HlgN0eHKPS>.
- Eran Malach. Auto-regressive next-token predictors are universal learners. *arXiv preprint arXiv:2309.06979*, 2023.
- Sean Michael McLeish, Arpit Bansal, Alex Stein, Neel Jain, John Kirchenbauer, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, Jonas Geiping, Avi Schwarzschild, and Tom Goldstein. Transformers can do arithmetic with the right embeddings. In *The 4th Workshop on Mathematical Reasoning and AI at NeurIPS'24*, 2024. URL <https://openreview.net/forum?id=cBFsFt1nDW>.
- Bhumika Mistry, Katayoun Farrahi, and Jonathon Hare. A primer for neural arithmetic logic modules. *Journal of Machine Learning Research*, 23:1–61, 2022. URL <https://www.jmlr.org/papers/volume23/21-0211/21-0211.pdf>.
- Arvind Neelakantan, Quoc V Le, and Ilya Sutskever. Neural programmer: Inducing latent programs with gradient descent. In *International Conference on Learning Representations (ICLR)*, 2016. URL <https://arxiv.org/abs/1511.04834>.
- Rodrigo Nogueira, Zhiying Jiang, and Jimmy Lin. Investigating the limitations of transformers with simple arithmetic tasks. *arXiv preprint arXiv:2102.13019*, 2021. URL <https://arxiv.org/abs/2102.13019>.
- Roman Novak, Lechao Xiao, Jiri Hron, Jaehoon Lee, Alexander A. Alemi, Jascha Sohl-Dickstein, and Samuel S. Schoenholz. Neural tangents: Fast and easy infinite neural networks in python. In *International Conference on Learning Representations*, 2020. URL <https://github.com/google/neural-tangents>.
- Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models, 2022. URL <https://openreview.net/forum?id=iedYJm92o0a>.
- Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022. URL <https://arxiv.org/abs/2201.02177>.
- Jorge Pérez, Pablo Barceló, and Javier Marinkovic. Attention is turing-complete. *Journal of Machine Learning Research*, 22(75):1–35, 2021. URL <https://dl.acm.org/doi/pdf/10.5555/3546258.3546333>.
- Scott Reed and Nando de Freitas. Neural programmer-interpreters. In *International Conference on Learning Representations (ICLR)*, 2016. URL <https://arxiv.org/abs/1511.06279>.
- Anian Ruoss, Grégoire Delétang, Tim Genewein, Jordi Grau-Moya, Róbert Csordás, Mehdi Bennani, Shane Legg, and Joel Veness. Randomized positional encodings boost length generalization of transformers. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting*

- of the Association for Computational Linguistics (*Volume 2: Short Papers*), pages 1889–1903, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-short.161. URL <https://aclanthology.org/2023.acl-short.161/>.
- David Saxton, Edward Grefenstette, Felix Hill, and Pushmeet Kohli. Analysing mathematical reasoning abilities of neural models. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=H1gR5iR5FX>.
- Ruoqi Shen, Sébastien Bubeck, Ronen Eldan, Yin Tat Lee, Yuanzhi Li, and Yi Zhang. Positional description matters for transformers arithmetic, 2023. URL <https://arxiv.org/abs/2311.14737>.
- Jack Sherman and Winifred J. Morrison. Adjustment of an inverse matrix corresponding to a change in one element of a given matrix. *The Annals of Mathematical Statistics*, 21(1):124–127, 1950. ISSN 00034851. URL <http://www.jstor.org/stable/2236561>.
- Hava Siegelmann and Eduardo Sontag. On the computational power of neural nets. *Journal of Computer and System Sciences*, 50:132–150, 1995. URL <https://www.sciencedirect.com/science/article/pii/S0022000085710136>.
- Andrew Trask, Felix Hill, Scott E Reed, Jack Rae, Chris Dyer, and Phil Blunsom. Neural arithmetic logic units. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/file/0e64a7b00c83e3d22ce6b3acf2c582b6-Paper.pdf.
- Yan Wang, Xiaojiang Liu, and Shuming Shi. Deep neural solver for math word problems. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 845–854, 2017. URL <https://aclanthology.org/D17-1088/>.
- Colin Wei, Yining Chen, and Tengyu Ma. Statistically meaningful approximation: a case study on approximating turing machines with transformers. *Advances in Neural Information Processing Systems*, 35: 12071–12083, 2022a. URL <https://dl.acm.org/doi/10.5555/3600270.3601147>.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022b. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf.
- Greg Yang. Tensor programs II: Neural tangent kernel for any architecture, 2020. URL <https://arxiv.org/abs/2006.14548>.
- Greg Yang and Etai Littwin. Tensor programs IIb: Architectural universality of neural tangent kernel training dynamics, 2021. URL <https://arxiv.org/abs/2105.03703>.
- Liu Yang, Kangwook Lee, Robert D Nowak, and Dimitris Papailiopoulos. Looped transformers are better at learning learning algorithms. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=HHbRxoDTxE>.
- Wojciech Zaremba and Ilya Sutskever. Learning to execute. *arXiv preprint arXiv:1410.4615*, 2014. URL <https://arxiv.org/abs/1410.4615>.
- Yongchao Zhou, Uri Alon, Xinyun Chen, Xuezhi Wang, Rishabh Agarwal, and Denny Zhou. Transformers can achieve length generalization but not robustly. In *ICLR 2024 Workshop on Mathematical and Empirical Understanding of Foundation Models*, 2024. URL <https://openreview.net/forum?id=DWkWIh3vFJ>.